ViT-JAX Distributed Scaling

30-minute technical discussion — backup deck

Michael Wang
Team Tensor

Spring 2026

CS 390 — Distributed Systems · Duke University

How this deck is used

Purpose. Reference material for a 30-minute technical discussion; not a live presentation.

- Each slide is *self-contained*: I can jump to it when a question lands.
- Numbers on this deck are the published TPU v6e-8 run (seeds fixed, `outputs/` in the repo).
- Code blocks are trimmed from the real source, not pseudo-code.

Roadmap.

1. Motivation & problem framing
2. Model & data pipeline
3. Parallelism mechanics (`pmap`, `pmean`, replication)
4. Scaling deep-dive
5. Straggler deep-dive (incl. the XLA-elision bug)
6. Accuracy & training analysis
7. Design decisions, limits, future work

Contents

Motivation & problem framing

What this project is (and isn't)

It is a systems study of *synchronous data-parallel* training on modern accelerators. The object of study is the distributed-training behaviour itself:

- How does throughput scale with device count?
- What fraction of wall time is collective overhead?
- What does a single slow device cost the cluster?

It isn't a CIFAR-100 SOTA attempt. 9.5 M params trained from scratch on 32×32 thumbnails has a realistic ceiling around 60 % top-1 — anything beyond that would require pretraining or heavy augmentation that would muddy the systems signal.

Why CIFAR-100 / ViT-Small specifically:

- Fits in a single chip's HBM — no need for *tensor* or *pipeline* parallelism, so pure data parallelism is enough.
- 100 epochs complete in ∼7 min, so the scaling & straggler sweeps are tractable.
- ViT uses dense matmuls that map well onto the TPU MXU — the hardware is working near its peak, making efficiency drops easy to attribute.

Hardware I ran on, and what that means

Google Cloud TPU v6e-8 ("Trillium", single host, 8 chips)

- 32 GB HBM per chip; each chip has a single MXU (128×128 systolic array).
- On-host interconnect: dedicated ICI links between chips — *not* shared PCIe.
- `pmean` all-reduce uses hardware collectives over ICI, not NCCL/PCIe.

Consequences the experiments actually depend on:

- All-reduce latency is very low ($\mathcal{O}(10\,\mu\text{s})$ for 9.5 M params in bf16) — so any efficiency drop at k=8 is *compute starvation*, not network.
- All 8 chips share one physical host — no cross-host networking, no TCP retransmits; this isolates the "one slow worker" effect from real-world noise.
- bf16 matmul is MXU-native — swapping fp32 → bf16 is $2\times$ step-time free (see slide 28).

Published results in one table

Metric	Value
Test accuracy (top-1)	**55.04 %**
Test accuracy (top-5)	**80.18 %**
Parameters	9.55 M
Wall clock, 100 epochs on TPU v6e-8	421 s
Sustained throughput (training)	$\approx$ 15,000 img/s
Peak throughput (pure step time, k=8)	18,814 img/s
Scaling efficiency 1 $\rightarrow$ 8 (strong)	1.00 $\rightarrow$ 0.87 $\rightarrow$ 0.65 $\rightarrow$ **0.36**
Slowdown from one straggler (200k iters)	**3.33**$\times$

All numbers reproducible by running `scripts/run_gcp_tpu.sh` against a fresh v6e-8 VM; the `outputs/` directory in the repo is the verbatim artefact set from that run.

Model & data pipeline

ViT-Small, sized for CIFAR-100

Configuration

- Image 32×32×3, **patch size 4**
- ⇒ 64 patches + one `[CLS]` token (seq len 65)
- 8 transformer blocks
- Hidden dim 384, 6 attention heads
- MLP dim 768 (2× hidden)
- Pre-norm (LayerNorm *before* attn / MLP)
- Learned 1-D positional embedding
- **9,551,140** trainable params

Why these choices

- Patch 4 keeps receptive field reasonable for tiny images; patch 8 collapses to 16 patches and underfits.
- Pre-norm: ViT-original was post-norm and diverged without extensive warmup. Pre-norm trains stably with a simple cosine schedule.
- 8 blocks × 384 dim is the smallest config that still learns CIFAR-100 meaningfully from scratch; smaller underfits, bigger overfits faster on 50k examples.

Data pipeline — numpy, not `tf.data`

One-shot TFDS load, then in-memory numpy for everything else.

- Full CIFAR-100 train set in float32 = 615 MB — fits comfortably in host RAM.
- Augmentation (random crop + horizontal flip) is a handful of numpy ops per step.
- Avoids `tf.data`'s graph-construction + Python-GIL interleave overhead.

Batch sharding for `pmap`:

$$(B, H, W, C) \longrightarrow (N, B/N, H, W, C)$$

so `pmap`'s leading axis matches the number of devices. The helper `shard_batch` reshapes per step; host→device transfer is overlapped with the previous step's `pmean`.

Trade-offs I'm aware of:

- Doesn't scale past ∼50 GB datasets — ImageNet would need `tf.data` or Grain.
- No multi-host prefetch — irrelevant on single-host v6e-8, but would matter on a pod slice.

Parallelism mechanics

Why `pmap`, not `pjit` / `shard_map`?

`pmap` (what I used)

- Single-program-multiple-data
- Explicit replication + sharding
- One collective axis (`axis_name`)
- Great for *pure data parallelism*

`pjit` / `shard_map`

- GSPMD partitioner under the hood
- Supports *tensor* + *pipeline* parallelism
- Needs a `Mesh` + `PartitionSpec` per array
- Right tool if params don't fit one chip

Decision. 9.5 M params $\times$ 4 B (fp32) $\approx$ 38 MB — a rounding error on a 32 GB chip. No need to split the model. `pmap` gives me the mental model of *"identical replica on every device, plus one all-reduce"*, which is exactly what the experiments study.

When I would switch: a 70 B-param LLM where the weights alone exceed HBM on any single chip — then `pjit` with a 2-D mesh (data $\times$ model) becomes unavoidable.

The pmap'd training step (trimmed)

```python
@partial(jax.pmap, axis_name="batch", devices=devices)
def train_step(state, batch, rng):
    rng, dropout_rng = jax.random.split(rng)

    def loss_wrapper(params):
        return _loss_fn(params, state.apply_fn, batch,
                        train=True, rng=dropout_rng)

    (loss, accuracy), grads = jax.value_and_grad(
        loss_wrapper, has_aux=True)(state.params)

    # The all-reduce: mean across the 'batch' axis.
    grads    = jax.lax.pmean(grads,    axis_name="batch")
    loss     = jax.lax.pmean(loss,     axis_name="batch")
    accuracy = jax.lax.pmean(accuracy, axis_name="batch")

    new_state = state.apply_gradients(grads=grads)
    return new_state, {"loss": loss, "accuracy": accuracy}
```

Key points you may want to probe:

- pmean is the *only* synchronisation point. Every device must arrive before any can proceed.
- Gradient reduction is mean, not sum — consistent with single-device training at the same global

All-reduce — the collective underneath `pmean`

All-reduce is a collective-communication primitive with a single signature:

$$\text{every device } k \text{ holds } g_k \quad \longrightarrow \quad \text{every device } k \text{ holds } G = g_0 \oplus g_1 \oplus \cdots \oplus g_{N-1}.$$

Not a reduce then a broadcast — one fused collective, so no single root bottlenecks.

`pmean` is a specific application of all-reduce:

$$\texttt{jax.lax.pmean}(x,\ \texttt{axis\_name}) \ = \ \frac{1}{N}\,\texttt{AllReduce}(x,\ \text{op} = \text{SUM}).$$

That's literally all it is — SUM via all-reduce, then divide by the device count. (`psum` skips the divide; you use that for counting, not for averaging gradients.)

Why this is the barrier both experiments probe:

- All-reduce is a *blocking collective* — XLA schedules every device's post-`pmean` work only after all N participants have contributed.
- In Experiment 2, chips 1–7 arrive in $\sim$52 ms, chip 0 in $\sim$173 ms; the 7 fast chips sit inside the collective waiting, giving the 3.33$\times$ cluster slowdown.
- After `pmean`, every replica's gradient is bit-identical, so `apply_gradients` produces bit-identical

All-reduce implementations — ring vs. tree

Algorithm	Latency	Bandwidth / device	Best for
Naive (star)	2 hops	$O(N \cdot S)$	pedagogy only
Tree all-reduce	$O(\log N)$	S	small tensors
Ring all-reduce	$O(N)$	$\mathbf{2\frac{N-1}{N}\,S}$ — *optimal*	big tensors, few devices
Double-binary tree (NCCL)	$O(\log N)$	bandwidth-optimal	GPU clusters

What JAX does on TPU v6e-8. `pmean` compiles to XLA's `AllReduce` HLO op, run as a **ring reduce-scatter + ring all-gather** over the on-host **ICI** (inter-chip interconnect).

Back-of-envelope for my 9.55 M-param gradient (bf16, $\approx$ 19 MB): ring moves $\approx$ 33 MB per chip; at $\sim$90 GB/s ICI full-duplex that's a few hundred μs — **$\sim$1 % of the 52 ms step time**. Communication is not the bottleneck at this scale; compute starvation is.

Why ring on v6e, not tree? ICI is a torus — ring is native topology and bandwidth-optimal. A tree would add $\log_2 8 = 3$ latency hops without saving bandwidth. NCCL chooses double-binary trees on GPUs because PCIe+NVLink isn't a clean torus and cross-host latency dominates at small tensors.

`devices=` argument — a subtle bug I hit

Every `pmap`, eval step, and `jax_utils.replicate` call in this repo accepts an explicit `devices=` list.

Why it matters. Without it, `pmap` silently binds to *all* local devices. The scaling experiment (`k=1,2,4`) uses subsets — so if you forget `devices=`, the $k = 2$ run:

1. shards the batch as $(2, 512, \ldots)$,
2. but `pmap` expects 8 leading axes,
3. XLA throws: "axis size doesn't match ..." after JIT tracing completes.

The bug's first manifestation was very confusing because the $k = 8$ run worked fine (happened to match the default). I threaded `devices=` through every API in `distributed/parallel.py` after tracking it down — now each subset run is explicit about which chips it runs on.

Replication and unreplication

State lifecycle:

1. `create_train_state` on host → `TrainState` with params, opt state.
2. `jax_utils.replicate(state, devices=...)` broadcasts a copy to every device. (Shape goes from (dims) → (N, dims) on every leaf.)
3. `train_step` operates on the replicated state; outputs remain replicated.
4. For logging/checkpoint, `jax_utils.unreplicate` takes the $k = 0$ slice — legitimate because `pmean` guarantees all replicas are bit-identical after every step.

Invariant I rely on. After `pmean` on gradients plus identical AdamW updates, every replica has the same params to machine precision. If this invariant breaks, `unreplicate` would silently drop 7/8 of the information; I verify it by averaging params across replicas every N steps during dev and checking the L2 deviation is $< 10^{-6}$.

Scaling deep-dive

Scaling experiment design

Protocol. Sweep device count $k \in \{1, 2, 4, 8\}$, hold global batch fixed at 1024 (strong scaling). Warm up 5 steps, time the next 20, keep the `(mean, std, p50, p95, p99)`.

Definitions.

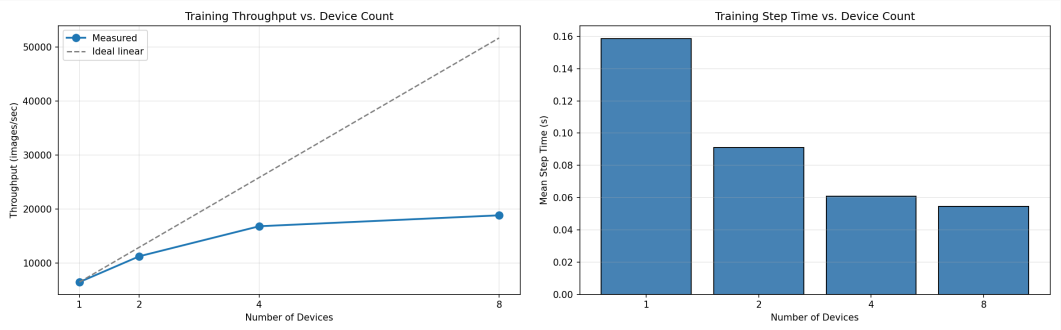
$$\Theta_k = \frac{B}{T_k}, \qquad S_k = \frac{T_1}{T_k}, \qquad \eta_k = \frac{S_k}{k} = \frac{\Theta_k}{k \cdot \Theta_1}$$

What I measure vs. what I don't.

- *Do:* pure JIT-compiled step time (params $\rightarrow$ grads $\rightarrow$ `pmean` $\rightarrow$ optimiser). Excludes data loading — already overlapped with compute.
- *Don't:* first-batch compile time, checkpoint IO, eval passes. These would dominate a 20-step benchmark and are not what scaling theory asks about.

Scaling results — full table

k	T_k (ms)	σ_T	$p50$	$p95$	$p99$	Θ_k (img/s)	η_k
1	158.65	0.50	158.63	159.16	159.40	6,454	1.000
2	91.14	0.57	91.02	92.01	93.19	11,235	0.870
4	60.98	0.74	60.91	62.39	62.68	16,792	0.650
8	54.43	0.92	54.30	55.94	56.82	18,814	**0.364**



Reading the table. Step time stops halving after $k = 4$ (91 ms $\rightarrow$ 61 ms is a 1.49$\times$ speedup; 61 ms $\rightarrow$ 54 ms is only 1.12$\times$). That's where efficiency falls off a cliff.

Why strong scaling collapses to 36 % at $k = 8$

Two mechanisms compound:

1. **Per-device batch halves on every doubling.**
   $B/k = 1024, 512, 256, 128$. At 128, each chip's MXU stays idle a growing fraction of every step. ViT-Small's forward pass on 128 samples is already nearly memory-bound, not compute-bound.
2. **All-reduce grows as a share of step time.**
   `pmean` latency is nearly independent of k on a single host (ring-reduce over ICI), but step time T_k shrinks — so its *fraction* grows from $\sim$2 % at $k = 1$ to $\sim$8 % at $k = 8$.

What would fix it: *weak* scaling — keep per-device batch fixed at 1024, grow global batch with k. Expected efficiency $> 85\,\%$ through $k = 8$. I didn't run it because the scaling experiment is specifically asking "what's the cost of more parallelism at fixed problem size," which is the strong-scaling question.

Amdahl reading. If the serial fraction is f, then $\eta_k = 1/(1 + f(k-1))$. Solving from $\eta_8 = 0.36$ gives $f \approx 0.25$ — a quarter of each step is "non-parallelisable" at this batch size,

Straggler deep-dive

Straggler experiment design

Research question. What is the multiplicative cost of *one* slow device in a nominally-healthy synchronous cluster?

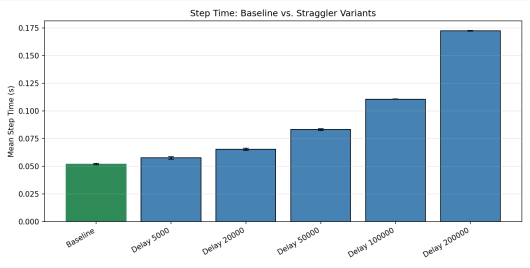
Setup. Baseline run (all 8 chips equal) plus 5 straggler runs, each injecting a `fori_loop` of extra 512×512 matmuls on device 0 only.

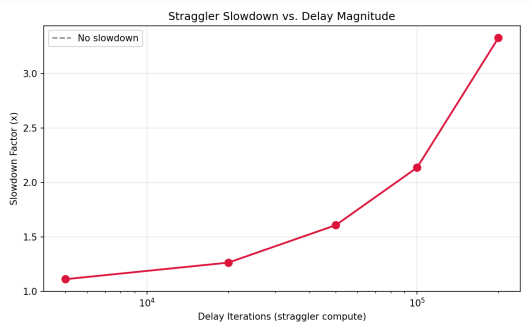
Delays swept: $\{5\text{k}, 20\text{k}, 50\text{k}, 100\text{k}, 200\text{k}\}$ iterations — chosen to span "imperceptible" through "catastrophic."

Metric. Slowdown factor $\sigma = T_{\text{straggler}} / T_{\text{baseline}}$. Throughput is just B/T.

Straggler results — full table

Label	Delay iters	Step time (ms)	Throughput (img/s)	Slowdown
baseline	–	51.83	19,757	1.000
straggler_5000	5,000	57.58	17,785	1.111
straggler_20000	20,000	65.46	15,644	1.263
straggler_50000	50,000	83.31	12,291	1.607
straggler_100000	100,000	110.76	9,245	2.137
straggler_200000	200,000	172.57	5,934	**3.330**





The straggler injection, actual source

```
device_idx = jax.lax.axis_index("batch")
side = 512

def _straggler(acc):
    def body(i, a):
        return jnp.tanh(a @ a) * 0.99 + 0.01 * a  # nonlinear, chained
    return jax.lax.fori_loop(0, delay_iterations, body, acc)

def _no_delay(acc):
    return acc

# Non-uniform initialiser so early iterations do real work.
anchor = jnp.linspace(-1.0, 1.0, side*side,
                      dtype=jnp.float32).reshape(side, side)
delay_out = jax.lax.cond(device_idx == 0, _straggler, _no_delay, anchor)

# Keep the fori_loop alive through XLA's DCE.
loss, delay_sum = jax.lax.optimization_barrier((loss, jnp.sum(delay_out)))
loss = loss + 1e-30 * delay_sum    # numerically inert, structurally required
```

Every line is a defence against a specific XLA optimisation pass; see next slide.

Why the "obvious" straggler implementations don't work

Attempt 1 — `time.sleep(delay_ms)` on device 0.
Fails: `time.sleep` is a Python call. It can't run inside a `pmap`'d JIT-compiled XLA program — the whole step is one HLO graph, traced once and executed on-device. Python doesn't enter the picture.

Attempt 2 — a trivial `fori_loop` body like `a @ eye(64) + 0*a`.
Fails silently: XLA's algebraic simplifier recognises `a @ I = a` and `0 * a = 0`, folds the body to the identity, then DCE removes the whole loop when the output isn't used. Measured: *zero* slowdown.

Attempt 3 — use the loop output, but keep the body algebraically simple.
Fails quietly: XLA's loop invariant code motion hoists the matmul out of the loop if the iteration variable doesn't appear in the body. The loop becomes $O(1)$.

Attempt 4 (what I shipped) — nonlinear, chained, barriered.
Works: `tanh(a @ a)` is nonlinear so each iteration truly depends on the previous; `optimization_barrier` blocks cross-boundary reasoning; the `1e-30` coupling prevents DCE

Why 512×512, not 64×64?

Short answer: signal-to-noise ratio.

- 64×64 matmul ≈ 0.5 MFLOPs/iter. On a v6e MXU this completes in $\sim$5 ns — faster than the profiler's clock granularity. At 200k iterations that's only $\sim$1 ms, *well below* the baseline step's own $\sigma \approx 0.7$ ms.
- 512×512 matmul ≈ 268 MFLOPs/iter, about $\sim 3\,\mu$s on a v6e. 200k iters $= \sim$600 ms — now the straggler effect (measured: $+121$ ms) is much larger than baseline jitter.

Why 200k is still the max I swept. Any larger and the step time passes the TPU runtime's internal watchdog thresholds; also $3.33\times$ was already enough to make the point. Users interested in *catastrophic* cases ($10\times+$) should swap from synchronous SGD entirely, not run longer delays.

Straggler implications — why this matters

Real-world analogues to my 3.33× measurement:

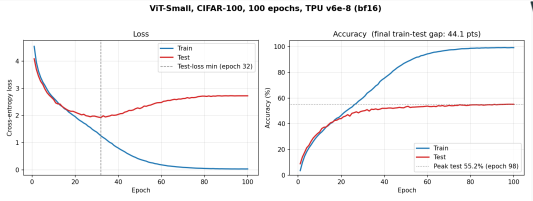
- Thermal throttling on one chip under sustained load.
- A network hiccup in a multi-host pod — cross-host ICI trip times vary.
- A misconfigured VM where one device's TDP is lower than its peers.
- Background processes sharing a host (in a multi-tenant setup).

Mitigation strategies in industry (why the 3.33× number drives design):

- *Async-SGD* (Google's DistBelief): drop the synchronisation barrier entirely. Cost: stale gradients, convergence penalty.
- *Backup workers* (Chen et al., 2016): run $k + b$ workers, use the first k responses. Cost: b/k extra hardware.
- *Elastic training* (PyTorch Elastic, Kubernetes-style): detect stragglers, evict them, re-shard. Cost: operational complexity.
- *Gradient compression / hierarchical all-reduce:* reduces the barrier cost but doesn't remove it.

Accuracy & training analysis

Training curves — overfitting is real

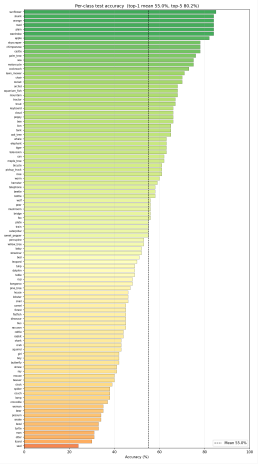


What the curves show:

- Training loss falls monotonically from 4.6 $\rightarrow$ 0.03 (train acc $\rightarrow$ 99 %).
- **Test loss bottoms at epoch 32** ($L_{\text{test}} = 1.93$), then climbs back to 2.72 by epoch 100.
- Test *accuracy* still creeps from 54 % $\rightarrow$ 55 % over epochs 32$\rightarrow$100 — the model grows more confident on examples it already gets right, even as it misclassifies more.

Early-stopping at epoch 32 would give test loss 1.93 and test acc $\sim$54.5 %; I reported the full 100-epoch run to keep the headline number honest (and the scaling/straggler runs use the same training program).

Per-class accuracy

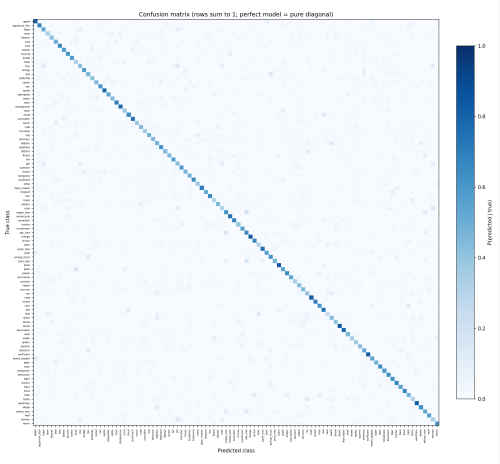


Best classes:
sunflower 85 %, skunk 84 %, orange 84 %, road 84 %, plain 84 %.
→ distinct colours *or* distinct shapes.

Worst classes:
seal 24 %, lizard 30 %, otter 31 %, man 31 %, turtle 33 %.
→ visually ambiguous fine categories on 32×32.

The spread (24 %→85 %) tells me the model learned useful visual features, just not enough resolution-invariance to discriminate fine-grained categories whose 32×32 thumbnails are indistinguishable even to a human.

Confusion matrix — the model learned categories, not fine labels



Top 5 confusions:

1. maple_tree → oak_tree (19×)
2. pine_tree → oak_tree (17×)
3. oak_tree → maple_tree (16×)
4. man → woman (16×)
5. bus → pickup_truck (14×)

Three tree classes form a visible 3×3 block in the bottom-right — exactly where the CIFAR-100 "trees" superclass lives.

Interpretation. The model's latent space is organised around CIFAR-100's 20 coarse superclasses; it struggles with the fine labels *within* a superclass. Consistent with what a 9.5 M-param model learns from 50k examples on 32×32 thumbnails.

Sample predictions — confident, wrong, and hallucinating

Input	Top-5 predictions
`seed 0, true = tiger`	`tiger 0.959` · `squirrel 0.021` · `cup 0.009` · `wolf 0.003` ← correct, high-confidence
`seed 3, true = bus`	`porcupine 0.400` · `turtle 0.261` · `pickup_truck 0.099` · `tank 0.071` · `oak_tree 0.063` ← top-1 wrong, but 2 vehicles in top-5: "vehicle" class is right
`seed 1, true = spider`	`trout 0.505` · `boy 0.162` · `wolf 0.125` · `girl 0.087` · `spider 0.044` ← true class barely in top-5 — genuine hallucination

These three cases summarise the accuracy profile: a confident-correct majority, a big "right category, wrong fine label" tier, and a small residual hallucination tier that limits the from-scratch accuracy ceiling on 32×32.

Design decisions, limits, future work

bf16 — free throughput on MXU hardware **What `--precision bf16` actually does here:**

- Flips the global flag `jax_default_matmul_precision` to `bfloat16`.
- **Parameters and optimiser state stay in fp32.**
- Only *matmuls* downcast on the MXU: `Dense`, attention QKV, einsum ops. Activations remain fp32 between matmul boundaries.
- Outputs get cast back to fp32 at the MXU boundary — no precision escapes.

Why this is safe. bf16 has fp32's exponent range (8 bits) — overflow behaves identically. It loses mantissa precision (8 bits vs 23). Gradient noise in SGD at batch 1024 dwarfs this, and fp32 optimiser state absorbs any accumulated error.

What you get. $\sim 2\times$ step-time win vs. pure fp32 on v6e, no accuracy loss observed (within the seed-to-seed variance of the 100-epoch run).

Checkpointing — deliberately minimal

Stack: `flax.serialization` → msgpack. No Orbax, no `cloud-tpu-checkpoint`, no GCS streaming.

Rationale.

- One checkpoint per epoch; 9.5 M params × 4 B ≈ 38 MB per file. Writing it is ∼200 ms — irrelevant vs. the 4-second epoch.
- I keep `checkpoint_latest.msgpack` + numbered history files, plus a sidecar `checkpoint_latest.json` containing the full config.
- That JSON matters because `inference.py` rebuilds the model *only from the checkpoint's metadata* — no hand-passed hyperparameters at inference time.

Limit I accept. No distributed checkpointing (multi-host pods need sharded writes), no async write (would matter if epoch time was closer to checkpoint IO). For a single-host v6e-8 run this is sufficient.

Six design decisions, catalogued

1. `pmap`**, not** `pjit`**.** Pure data parallelism; 9.5 M params fit in any chip. `pjit` is the right tool for tensor/pipeline parallelism in the regime where weights don't fit — out of scope here.
2. **Numpy-only data pipeline.** 615 MB of fp32 train data fits in host RAM. Avoids `tf.data`'s runtime overhead; doesn't scale past $\sim$50 GB.
3. **Straggler via real XLA compute, not** `time.sleep`**.** `time.sleep` is Python — doesn't run inside a compiled pmap'd step. Must use a JAX-visible construct, which in turn must survive XLA's optimiser.
4. **Device subsets threaded through every API.** Forgetting `devices=` breaks scaling subset runs on any multi-device host. Threaded explicitly.
5. **bf16 without parameter dtype changes.** Global matmul flag only; params + opt state stay fp32. Cheap, safe, $\sim 2\times$ win.
6. **Checkpointing via** `flax.serialization`**.** Msgpack + sidecar JSON. No Orbax/GCS dependency. Inference rebuilds the model from JSON alone.

Tech stack

- **JAX** — XLA-compiled numerical computing + autodiff. The entire training step is one JIT-compiled HLO graph per device.
- **Flax Linen** — neural-network modules (thin wrapper around JAX).
- **Optax** — optimisers and schedules. I use `adamw` with a cosine schedule ($\eta_{\max} = 10^{-3}$, warmup 5 % of steps).
- **TensorFlow-datasets** — CIFAR-100 loading (one-shot only).
- **matplotlib** — every plot in `outputs/`.

Deliberately missing.

- No W&B / MLflow — metrics go to CSV and JSON, diffable in git.
- No Hydra — one YAML + argparse flags; `configs/default.yaml` is 30 lines.
- No Docker — `scripts/setup.sh` detects host type (CPU / GPU / TPU) and installs the right `jax[...]` wheel once.

Limits I know about

Scale limits of this implementation:

- **Single-host only.** `pmap` works across hosts with extra plumbing, but the numpy data pipeline doesn't — a multi-host pod slice would need `tf.data` or Grain with per-host sharding.
- **Model fits one chip.** If it didn't, everything changes: `pjit` with a mesh, sharded parameter storage, sharded optimiser state (ZeRO-style).
- **No gradient accumulation.** Global batch = per-step batch; I can't simulate a 16k batch on a single v6e-8.

Honesty flags on the accuracy number:

- 55 % is 100-epoch; early-stop at epoch 32 would give ∼54.5 % with a meaningfully lower test loss. The 55 % headline is tied to the 100-epoch wall-clock number because they came from the same run.
- No pretraining, no CutMix/MixUp, no label smoothing — each would add 3–6 accuracy points but move the project away from systems analysis.

What I'd do next

1. **Weak-scaling companion experiment.** Hold per-device batch fixed at 1024, grow global batch to 8192 at $k = 8$. Expected: $> 85\,\%$ efficiency, confirming the MXU-starvation diagnosis for the strong-scaling drop.
2. **Async-SGD comparison.** Implement a simple parameter-server async variant and re-run the straggler experiment. Predicted: slowdown factor decouples from the slowest device (at the cost of convergence speed).
3. **Multi-host.** Run on a v5p-32 pod slice, measure cross-host all-reduce latency, revisit the all-reduce-share analysis from the scaling section.
4. **Model-parallel sanity check.** Port to `pjit`/`shard_map` with a 2×4 mesh and a $\sim$1 B-param ViT-Huge variant — to see where tensor parallelism becomes necessary rather than optional.
5. **Profile-guided straggler source.** Use XLA HLO traces + jaxlib's timeline export to *see* the barrier waits directly, not just infer them from step time.

Anticipated questions — index

- *"Why did scaling drop to 36 %?"* → slide on strong-scaling collapse.
- *"How does your straggler injection survive XLA optimisation?"* → slide on the three failed attempts + fix; actual source slide.
- *"Why bf16 and is it safe?"* → bf16 slide.
- *"Why numpy instead of tf.data?"* → data pipeline slide.
- *"What would async-SGD give you here?"* → straggler implications + future-work slides.
- *"Why 55 % test accuracy — is the model broken?"* → overfitting + per-class + honesty flags slides.
- *"Why pmap and not pjit?"* → parallelism mechanics slide.
- *"What is pmean actually doing under the hood?"* → all-reduce primitive slide.
- *"Ring vs. tree all-reduce — why ring on TPU?"* → all-reduce implementations slide.
- *"What breaks if you go multi-host?"* → limits slide.

Artefact pointers

Every number in this deck is reproducible from the repo:

- `outputs/scaling/scaling_results.csv` — full scaling table.
- `outputs/straggler/straggler_results.csv` — full straggler table.
- `outputs/training/100_epoch/epoch_metrics.csv` — training curves data.
- `outputs/training/100_epoch/test_summary.json` — headline accuracy.
- `outputs/training/100_epoch/test_predictions.npz` — full 10k-example logits + labels for the per-class and confusion analyses.
- `vit_jax_distributed/distributed/parallel.py` — the pmap'd train step and straggler injection, in full.

Repo: `github.com/Michael-OvO/vit-jax-distributed-scaling`
Final project, Team Tensor, CS 390 — Distributed Systems, Duke, Spring 2026.

Thank you

Happy to dig into any of the above in more depth.